

Games and AI Techniques

Kinematics - Steering Behaviors

What is AI?

- How would you define AI?

Thinking Humanly "The exciting new effort to make computers think ... <i>machines with minds</i>, in the full and literal sense." (Haugeland, 1985) "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning .. ." (Hellman, 1978)	Thinking Rationally "The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly "The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The study of how to make computers do things at which, at the moment, people are better." (Rich and Knight, 1991)	Acting Rationally "Computational Intelligence is the study of the design of intelligent agents." (Poole <i>et al</i>, 1998) "AI ... is concerned with intelligent behavior in artifacts." (Nilsson, 1998)
Figure 1.1 Some definitions of artificial intelligence, organized into four categories.	

Steering Behaviors For Autonomous Characters

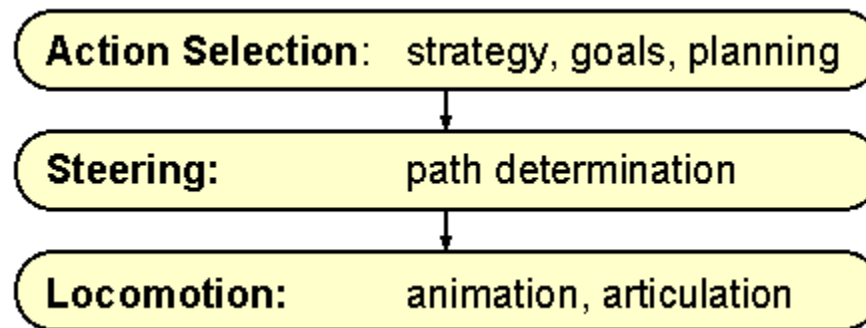
Craig W. Reynolds & AI4G Ian Millington and John Funge

Outline

- Introduction
- Simple Vehicle Model
- The Physics of The Model
- Steering behaviors
 - One or two characters behaviors
 - Group behaviors
- Combining Behaviors
- Conclusion

Introduction

- To navigate **autonomous characters** around their world in a life-like and improvisational manner in animation and games
- Hierarchy of motion behaviors

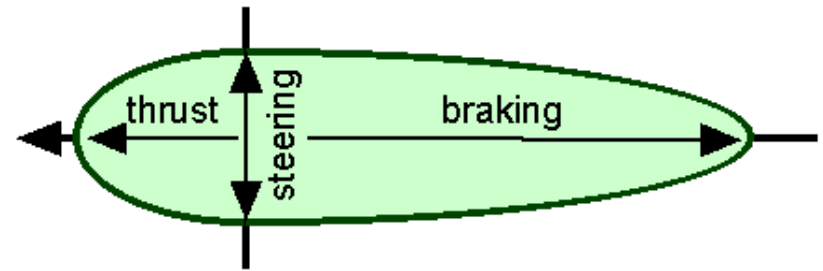


Movement

- Movement calculation often needs to interact
- with the “Physics” engine
 - Avoid characters walking through each other or through obstacles
- Traditional: kinematic movement (not dynamic)
 - Characters simply move (often at fixed speed) without regard to how physical objects accelerate or brake
- Newer approach:
 - Steering behaviors or dynamic movement (Craig Reynolds)
 - Characters accelerate and turn based on physics
 - Seek, flee, flocking

Simple Vehicle Model

- Simple Vehicle Model:
 - mass scalar
 - position vector
 - velocity vector
 - max_force scalar
 - max_speed scalar
 - orientation N basis vectors



Orientation Basics

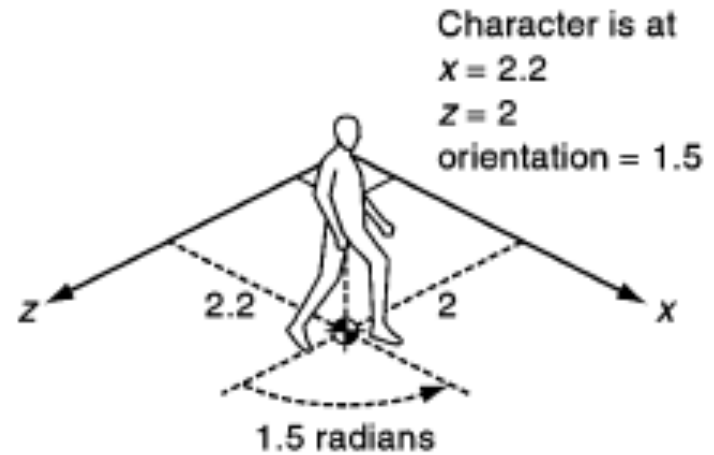
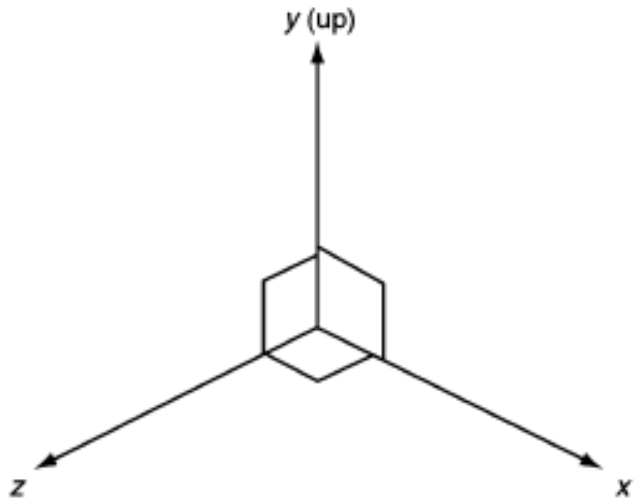
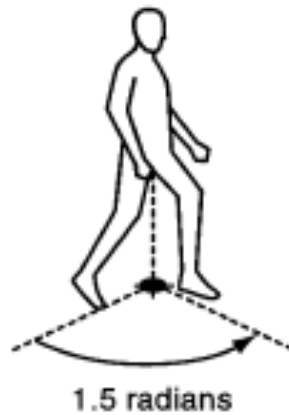


Figure 3.4 The positions of characters in the level

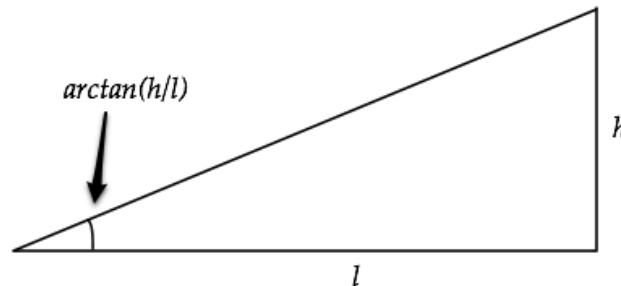
```
struct Static:  
    position    # a 2D vector  
    orientation # a single floating point value
```


Orientation Vector vs Scalar



Convenient to
represent
orientation as unit
vector (len = 1)

$\begin{bmatrix} 0.997 \\ 0.071 \end{bmatrix}$



Kinematics

- We describe a moving character by
 - Position
 - 2-dimensional vector
 - Orientation
 - 2-dimensional unit vector given by an angle, a single real value between 0 and 2π
 - Velocity
 - 2-dimensional vector
 - Rotation
 - 2-dimensional unit vector given by an angle, a single real value between 0 and 2π

Steering Behaviors

```
1 struct Kinematic:
2     position    # a 2 or 3D vector
3     orientation # a single floating point value
4     velocity    # another 2 or 3D vector
5     rotation    # a single floating point value
```

Steering behaviors operate with these kinematic data. They return accelerations that will change the velocities of a character in order to move them around the level. Their output is a set of accelerations:

```
1 struct SteeringOutput:
2     linear      # a 2 or 3D vector
3     angular     # a single floating point value
```

Basic Update Function

```
struct Kinematic:
```

```
... Member data as before ...
```

```
def update (steering, time): Time vs Frame
```

```
    # Update the position and orientation
```

```
    position += velocity * time
```

```
    orientation += rotation * time
```

Good idea to modulo this. Why? By what?

```
    # and the velocity and rotation
```

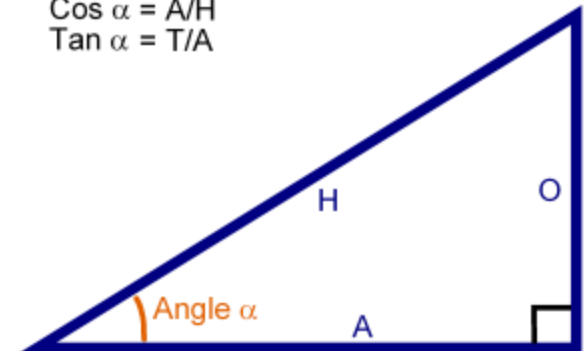
```
    velocity += steering.linear * time
```

```
    orientation += steering.angular * time
```

Update orientation

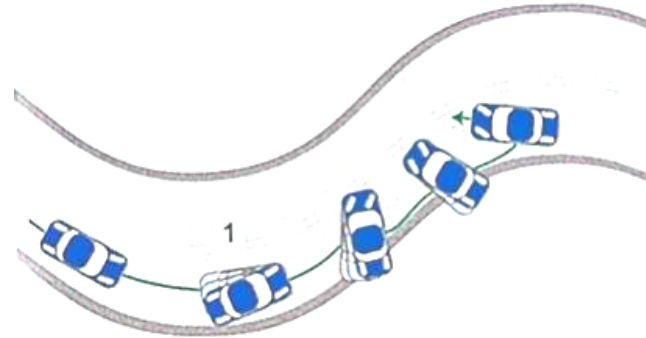
```
def getNewOrientation(currentOrientation, velocity):  
  
    # Make sure we have a velocity  
    if velocity.length() > 0:  
  
        # Calculate orientation using an arc tangent of  
        # the velocity components.  
        return atan2(-static.x, static.z)  
  
    # Otherwise use the current orientation  
    else: return currentOrientation
```

$$\begin{aligned}\sin \alpha &= O/H \\ \cos \alpha &= A/H \\ \tan \alpha &= T/A\end{aligned}$$



Constraint

- Because of its assumption of velocity alignment, this simple vehicle model cannot simulate effects such as **skids**, **spins** or **slides**.
- Furthermore this model allows the vehicle to turn when its speed is zero.



Kinematic Seek

- Input: character's & target's static data
- Output: velocity in direction from *char* to *targ*
 - $\text{velocity} = \text{target.position} - \text{character.position}$
- Normalize velocity to maximum velocity
- Can ignore orientation, or update (prev slide)

```
class KinematicSeek:
    # Holds the static data for the character and target
    character
    target

    # Holds the maximum speed the character can travel
    maxSpeed

    def getSteering():

        # Create the structure for output
        steering = new KinematicSteeringOutput()

        # Get the direction to the target
        steering.velocity =
            target.position - character.position

        # The velocity is along this direction, at full speed
        steering.velocity.normalize()
        steering.velocity *= maxSpeed

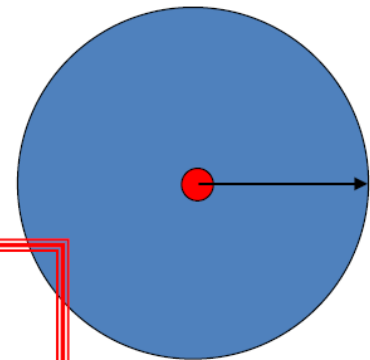
        # Face in the direction we want to move
        character.orientation =
            getNewOrientation(character.orientation,
                              steering.velocity)

        # Output the steering
        steering.rotation = 0
        return steering
```

Overshooting

- Seek with full velocity leads to overshooting
 - Arrival modification
 - Determine arrival target radius
 - Lower velocity within target for arrival

```
steering.velocity = target.position - character.position;  
if(steering.velocity.length() < radius)    {  
    steering.velocity /= timeToTarget;  
    if(steering.velocity.length() > MAXIMUMSPEED)  
        steering.velocity /= steering.velocity.length();  
}  
else  
    steering.velocity /= steering.velocity.length();
```



Arrival Circle:

Slow down if
you get here

Dynamic Seek

```
# Returns the desired steering output
def getSteering():

    # Create the structure to hold our output
    steering = new SteeringOutput()

    # Get the direction to the target
    steering.linear = target.position -
                      character.position

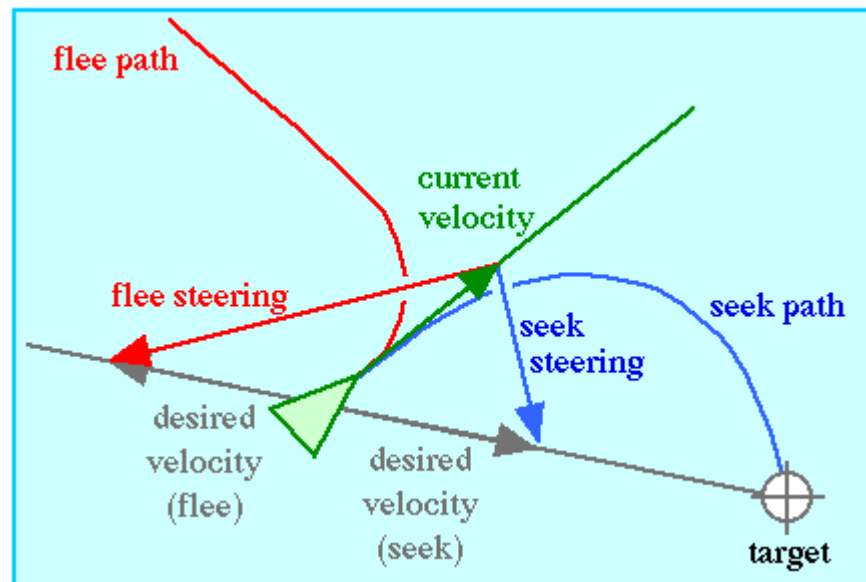
    # Give full acceleration along this direction
    steering.linear.normalize()
    steering.linear *= maxAcceleration

    # Output the steering
    steering.angular = 0
    return steering
```

No rotation but can
change this if needed as before

Seek and Flee

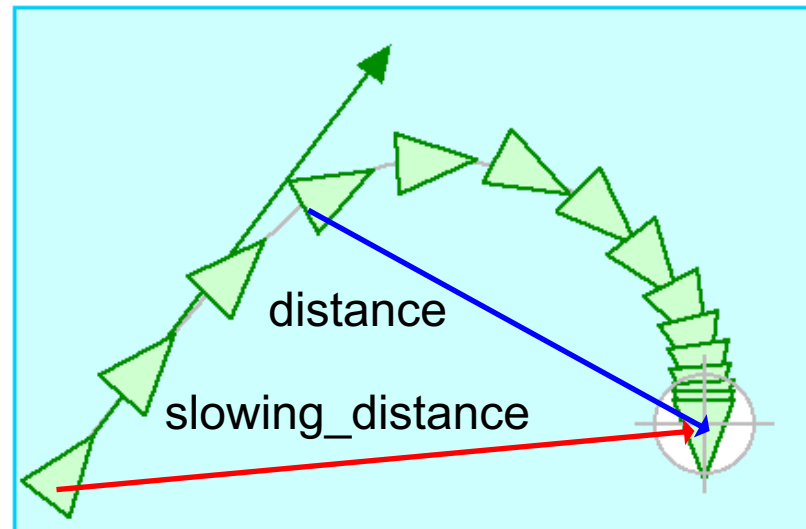
- <http://www.red3d.com/cwr/steer/SeekFlee.html>
- **Seek** (or pursuit of a static target)
 - **desired_velocity** = $\text{normalize}(\text{position} - \text{target}) * \text{max_speed}$
 - **steering** = **desired_velocity** – velocity
- **Flee** is simply the inverse of seek.
 - **steering** = velocity – **desired_velocity**



Arrival

- <http://www.red3d.com/cwr/steer/Arrival.html>
- **target_offset** = target – position
distance = length (target_offset)
ramped_speed = $\text{max_speed} * (\text{distance} / \text{slowing_distance})$
clipped_speed = minimum (ramped_speed, max_speed)
desired_velocity = $(\text{clipped_speed} / \text{distance}) * \text{target_offset}$
steering = desired_velocity - velocity

slowing_distance:
The distance at which
slowing begins.



Overshooting (orbit)

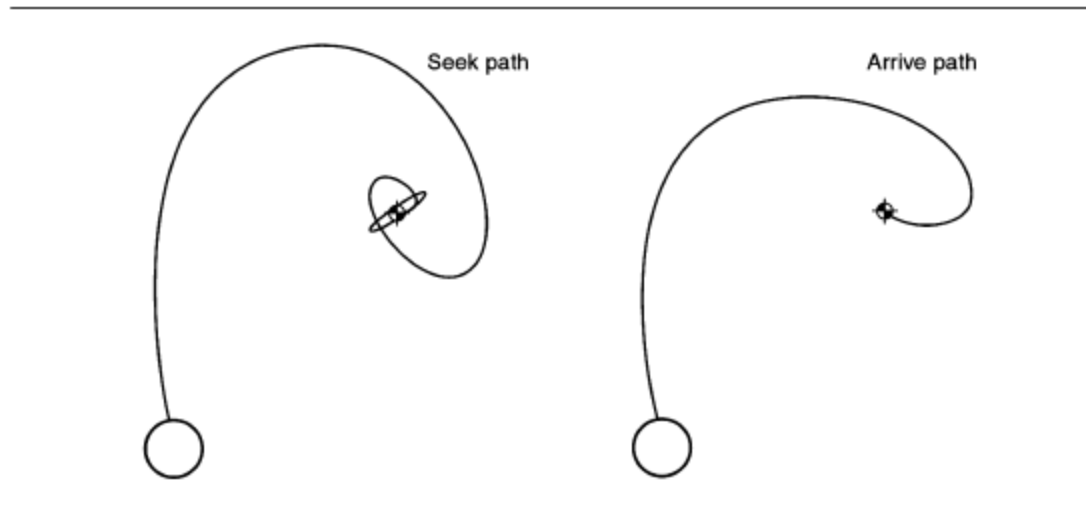
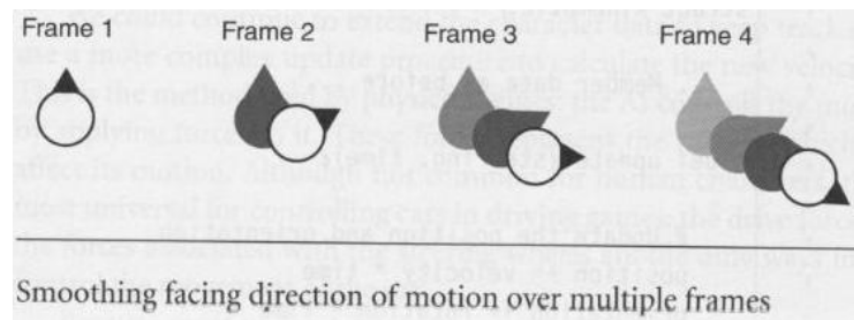


Figure 3.9 Seeking and arriving

Facing

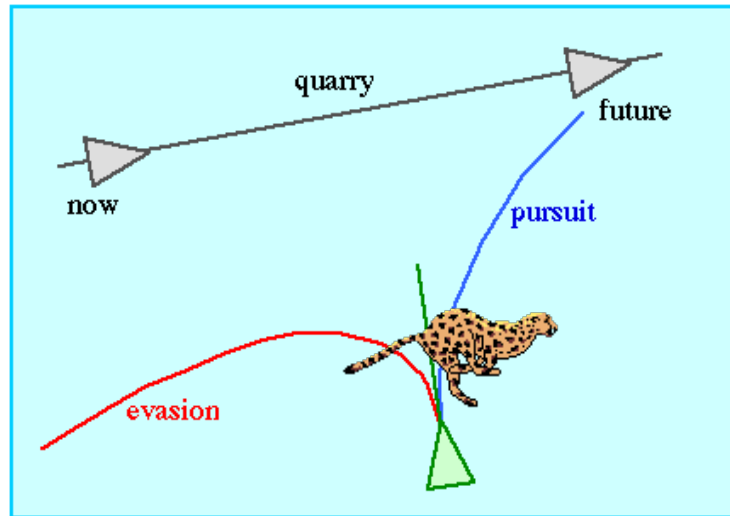
- Motion & facing need not be coupled
- Many games simplify & force character orientation to be in direction of the velocity
 - Instant (can be awkward)
 - Smoothing



Millington Fig 3.6

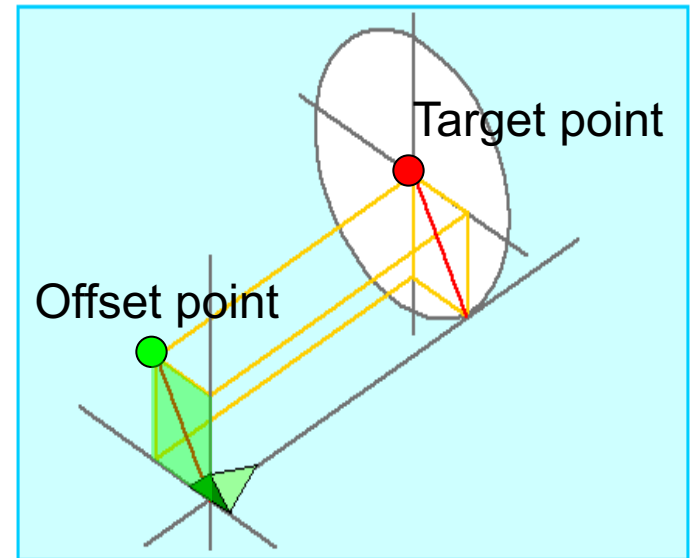
Pursuit and Evasion

- <http://www.red3d.com/cwr/steer/PursueEvade.html>
- **Pursuit** is similar to **seek** except that the quarry (target) is another moving character.
 - Assume the quarry will not turn during the prediction interval T .
 - Future position can be obtained by **scaling its velocity by T** and **adding that offset to its current position**.



Offset Pursuit

- Steering a path which passes near, but not directly into a moving target.
- **Localize** the predicted target location (character's local coordinate space) **project** the local target onto the character's *side-up* plane, normalize that lateral offset, **scale** it by $-R$, **add** it to the local target point, and **globalize** that value.
- Use **seek** behavior to approach that offset point.



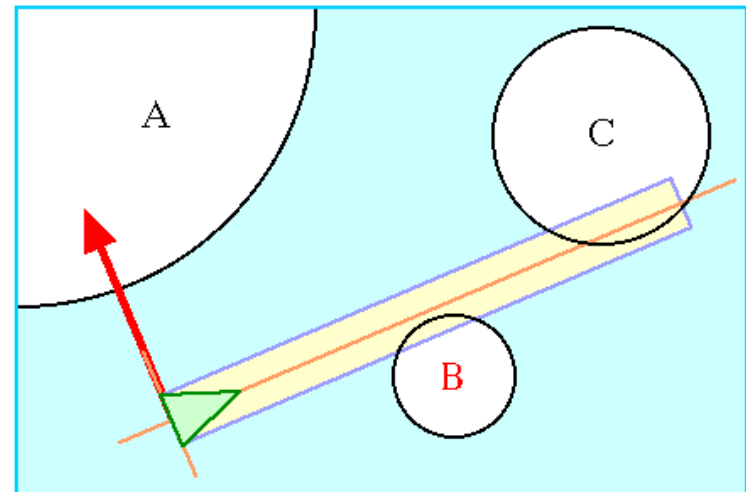
Obstacle Avoidance

- <http://www.red3d.com/cwr/steer/Obstacle.html>
- The goal of the behavior is to keep an **imaginary cylinder** of **free space** in front of the character.

The value returned from obstacle avoidance

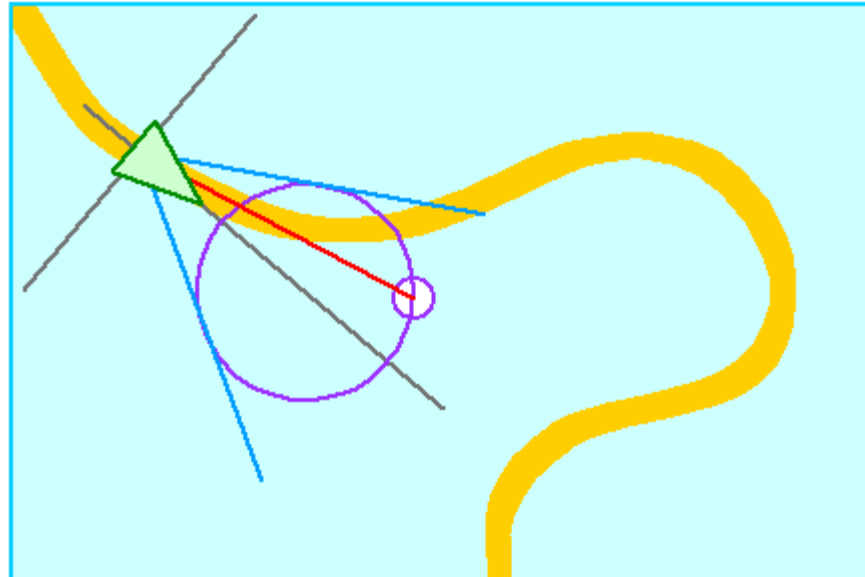
(a) the steering value to avoid **the most threatening obstacle**

(b) if **no collision is imminent**, a special value (a *null* value, or the zero vector) to indicate that no corrective steering is required at this moment.



Wander

- <http://www.red3d.com/cwr/steer/Wander.html>
- The steering direction is represented by a red vector.
- The big circle in figure constraint the steering.
- The small circle constraint the random offset of the steering.

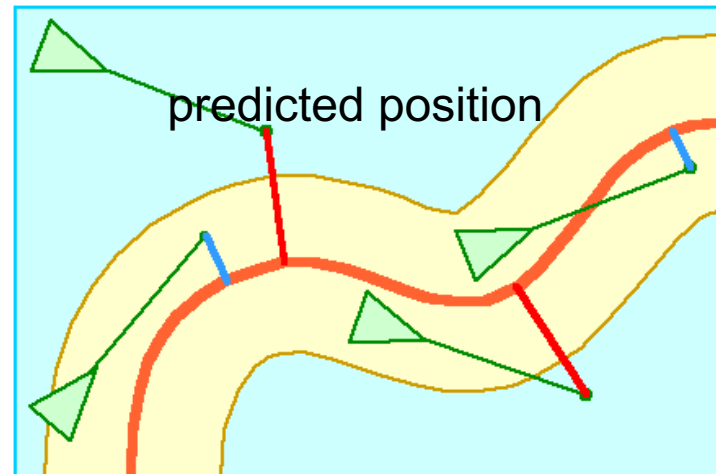


Path following

- <http://www.red3d.com/cwr/steer/PathFollow.html>
- To move a character along the path while staying within the specified radius of the spine.
- **Projection distance** is less than the path radius no corrective steering is required.
- Otherwise, **Seeking** towards the on-path projection of the predicted future position.

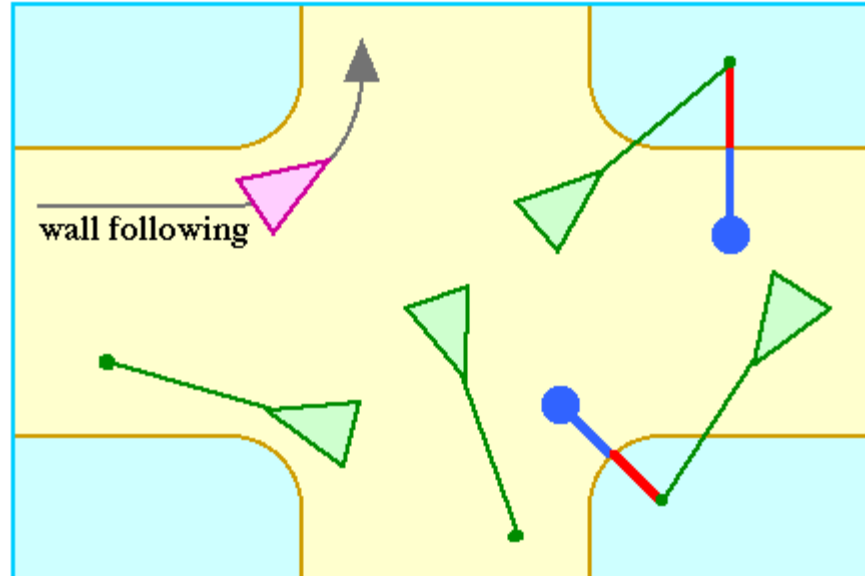
Projection distance

from the predicted position to
the nearest on-path point.



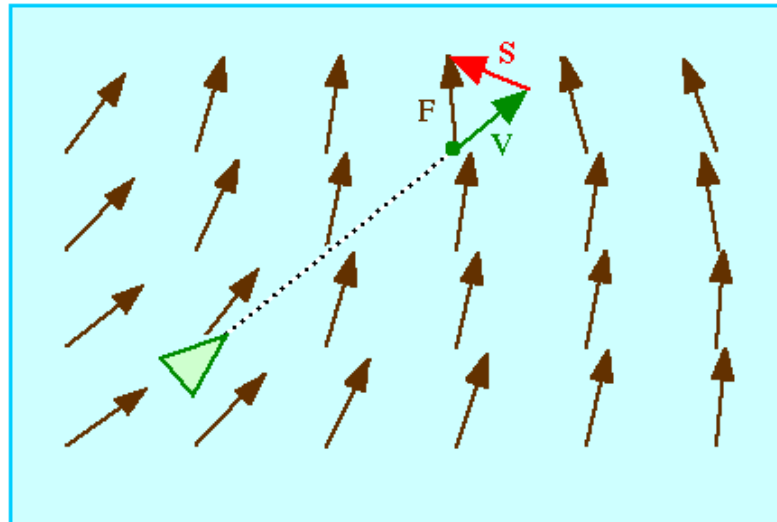
Wall Following and Containment

- <http://www.red3d.com/cwr/steer/Wall.html>
- <http://www.red3d.com/cwr/steer/Containment.html>
- Wall following can be implemented by **offset pursuit**.
- Containment can be accomplished by using **seek** with an inside point of the container.



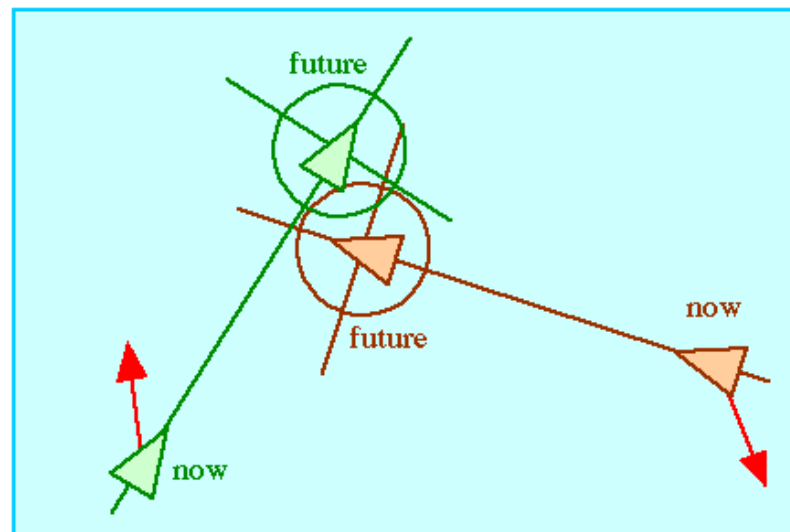
Flow Field Following

- <http://www.red3d.com/cwr/steer/FlowFollow.html>
- The **future position** of a character is estimated and the flow field is sampled at that location. This **flow direction (vector F)** is the “**desired velocity**” and the **steering direction (vector S)** is simply the difference between the current velocity (vector V) and the desired velocity.



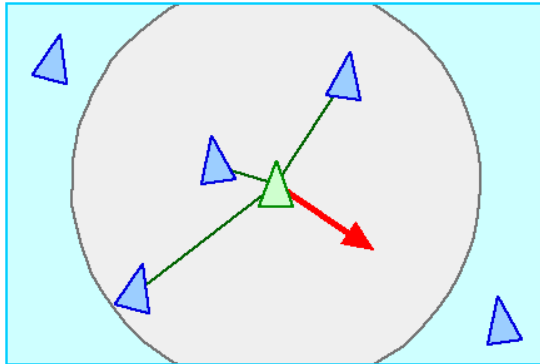
Unaligned Collision Avoidance

- The character steers to avoid the site of the predicted collision.
- If all nearby characters are *aligned*, a less complicated strategy can be used, see **separation** below.

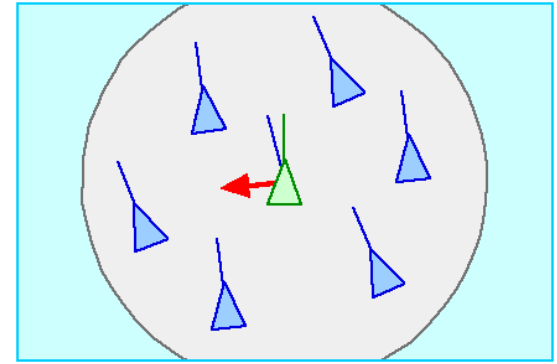
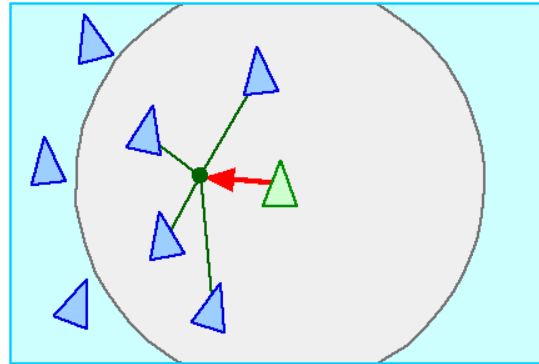


Group Behavior

- **Separation**, **cohesion**, and **alignment**, relate to groups of characters.
- the steering behavior determines how a character reacts to other characters in its local *neighborhood*.



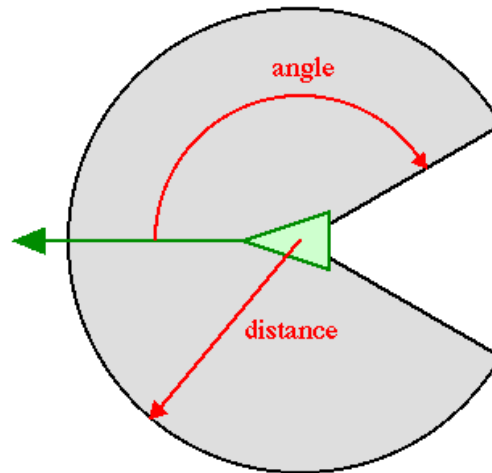
Repulsive force is computed by subtracting the positions of our character and the nearby character



Average velocity is the “desired velocity”

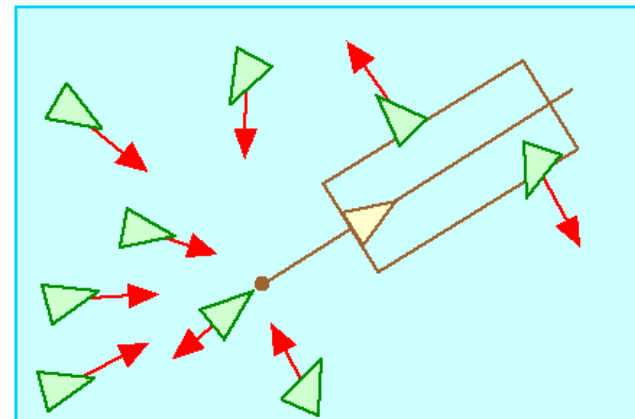
Neighborhood

- Local neighborhood of characters
 - the neighborhood is specified by a *distance* which defines when two characters are “nearby”, and an *angle* which defines the character’s perceptual “field of view”



Leader Following

- <http://www.red3d.com/cwr/steer/LeaderFollow.html>
- If a follower finds itself in a rectangular region in front of the leader, it will steer laterally away from the leader's path.
- Otherwise, the **arrival** target is a **point offset slightly behind the leader**.
- the followers use **separation** behavior to prevent crowding each other.



Combining Behaviors

- Combining behaviors can happen in two ways:
 - Switch
 - Blending
- The most straightforward is simply to compute each of the component steering behaviors and **sum** them together, possibly with a **weighting factor** for each of them.

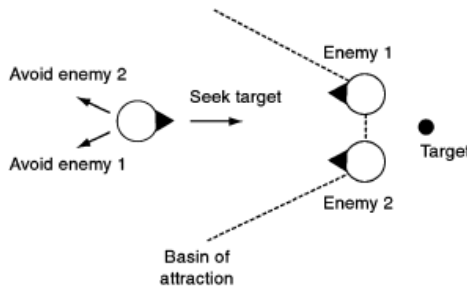


Figure 3.34 A stable equilibrium

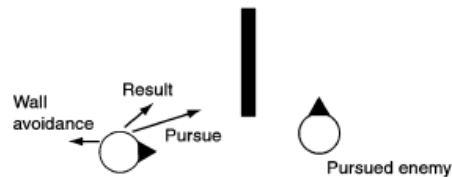


Figure 3.35 Can't avoid an obstacle and chase

Combining Behaviors Demo

- Crowd Path Following
- Leader Following
- Unaligned Collision Avoidance
- Queuing (at a doorway)
- Flocking (combining: *sepgration, alianment, cohesion*)

